

On-Device RAG with Phi-3.5-mini-instruct: CPU vs CUDA Accelerator Performance

Roland Vaatmann

December 26, 2025

Abstract

This report presents a local/on-device Retrieval-Augmented Generation (RAG) command-line application using Phi-3.5-mini-instruct (ONNX INT4 AWQ) and compares inference performance across two accelerators: CPU and NVIDIA CUDA. A fixed set of five questions from the NASA Systems Engineering Handbook is used to evaluate model load time, time-to-first-token (TTFT), total generation time, and effective throughput (tokens/second). Results show large reductions in TTFT and total generation time on CUDA versus CPU for long RAG prompts.

1 Objective

Build a demo CLI RAG application that runs Phi-3.5-mini-instruct locally, answers questions grounded in an external knowledge base, accelerates inference on two different accelerators (CPU and GPU), and reports relevant metrics (model load, TTFT, throughput, etc.) for performance analysis.

2 System Setup

- CPU: AMD Ryzen 9 9800X3D
- GPU: NVIDIA GeForce RTX 3080
- Model runtime: ONNX Runtime GenAI
- LLM: Phi-3.5-mini-instruct (ONNX INT4 AWQ variants)
- Knowledge base document: *NASA Systems Engineering Handbook* (PDF)

3 RAG Pipeline Overview

The CLI implements a standard RAG flow:

1. **Ingest:** PDF is chunked into overlapping text segments.
2. **Embed:** Each chunk is embedded with a sentence-transformer encoder.
3. **Index:** Embeddings are stored in an approximate nearest-neighbor index (HNSW).
4. **Retrieve:** For a user question, the top- k most similar chunks are retrieved.
5. **Generate:** Retrieved context is injected into a Phi chat prompt and answered by the LLM.

4 Evaluation Questions

The following five questions were used for consistent measurement:

1. According to the handbook, what is the difference between product verification and product validation?
2. According to the handbook, what are Measures of Effectiveness (MOEs) and how are they related to stakeholder expectations / mission success criteria?
3. According to the handbook, what are the four system design processes, and how do they relate iteratively/recursively?
4. According to the handbook, what are the typical outputs of the Stakeholder Expectations Definition process?
5. According to the handbook, what is the objective of systems engineering regarding designing, building, and operating a system in a cost-effective way, and what factors must be balanced?

5 Metrics

For each run, the CLI prints:

- `device_type`: CPU or CUDA.
- `model_load_s`: model initialization + load time.
- `prompt_tokens`: tokens in the final RAG prompt (question + retrieved context).
- `new_tokens`: generated output tokens.
- `ttft_s`: *normal TTFT*, measured from starting generator prefill (`append_tokens`) through the first generated token.
- `gen_total_s`: total LLM time including prefill and decoding until completion.
- `tok/s`: effective throughput computed as `new_tokens / gen_total_s`.

6 Results (Averages over 5 Questions)

Table 1 reports means across the five-question set. “Speedup” is computed as CPU/CUDA for latency metrics (lower is better), and CUDA/CPU for throughput (higher is better).

Metric	CPU Avg	CUDA Avg	Speedup
model_load_s (s)	2.199	1.534	1.43
prompt_tokens	1868.0	1868.0	–
new_tokens	209.8	200.8	–
ttft_s (s)	14.629	0.466	31.36
gen_total_s (s)	30.615	1.807	16.94
tok/s	6.824	110.318	16.17

Table 1: Mean performance across five NASA handbook questions. TTFT and total generation time include prompt prefill; tok/s is effective throughput including prefill.

7 Discussion

Prompt length impact. The average prompt length was 1868 tokens due to retrieved context. Long prompts increase prefill cost, which is especially expensive on CPU and is reflected in the large CPU TTFT (14.6s average).

Accelerator comparison. CUDA substantially reduces both prefill and decode time, producing a large improvement in TTFT ($31\times$ faster) and total LLM time ($17\times$ faster) for the tested RAG workload. Effective throughput improves from 6.82 tok/s (CPU) to 110.32 tok/s (CUDA).

Practical takeaway. For interactive RAG usage, TTFT is often the dominant user-perceived metric. The GPU accelerator provides a large advantage for responsiveness and overall throughput under long-context conditions.

8 Conclusion

A local RAG CLI application was implemented using Phi-3.5-mini-instruct (ONNX INT4 AWQ) and evaluated on two accelerators. Across five handbook-based questions, CUDA achieved dramatically lower TTFT and total generation time, and significantly higher effective throughput compared to CPU. These results support the conclusion that GPU acceleration is strongly beneficial for long-context RAG workloads.